DATA SCIENCE UC
FACULTAD DE MATEMÁTICAS

Herramientas Computacionales para Data Science

# Naive Bayes y clasificación probabilística

Sesión 40 · 1 de octubre de 2026

Eloy Alvarado Narváez

# Ahora conocemos la clase que queremos predecir

**En los vinos: buscar grupos**
Comparamos perfiles químicos para descubrir vinos parecidos. Los grupos eran un resultado del análisis.

**En los correos: aprender una clase**
Tenemos mensajes ya etiquetados como spam o legítimos. Buscamos señales que permitan clasificar el siguiente correo.

Usaremos $Y = 1$ para **spam** y $Y = 0$ para **legítimo**. Las entradas serán características del mensaje, como las palabras que aparecen.

**La pregunta de hoy es cómo construir una probabilidad de spam a partir de ejemplos etiquetados.**

# Construir un filtro que podamos explicar

Un correo contiene *oferta*. Queremos saber qué probabilidad de spam asignarle y si conviene bloquearlo.

| Paso | Pregunta que responderemos |
|---|---|
| Aprender de correos | ¿Qué señales aparecen dentro de cada clase? |
| Calcular y comprobar | ¿Cómo se combinan y qué tan fiables son las probabilidades? |
| Tomar una decisión | ¿Cuándo conviene bloquear, considerando los errores? |

**Primero seguiremos las cuentas en 100 correos construidos; después aplicaremos el mismo proceso a los 4601 correos de Spambase.**

# Tres preguntas distintas

| Nombre | Pregunta en el ejemplo | Qué observamos |
|---|---|---|
| Previa | ¿Qué fracción de los correos es spam? | La clase |
| Condicional | Entre los spam, ¿qué fracción contiene *oferta*? | La palabra dentro de la clase |
| Posterior | Entre los correos con *oferta*, ¿qué fracción es spam? | La clase al conocer la palabra |

**Bayes permite pasar de lo que sabemos de cada clase a una probabilidad para el correo que estamos mirando.**

# Un ejemplo con 100 correos

| Clase | Con “oferta” | Sin “oferta” | Total |
|---|---|---|---|
| Legítimo (0) | 8 | 72 | 80 |
| Spam (1) | 16 | 4 | 20 |
| Total | 24 | 76 | 100 |

La palabra aparece en 16 de los 20 spam y en 8 de los 80 correos legítimos.

**Entre los 24 correos que contienen “oferta”, 16 son spam.**

Ejemplo construido para los cálculos de las diapositivas 5–25; no son conteos de Spambase.

# La dirección del condicionamiento

$$\mathrm{P}(\text{oferta} \mid \text{spam}) = \frac{16}{20} = 0.80, \qquad \mathrm{P}(\text{oferta} \mid \text{legítimo}) = \frac{8}{80} = 0.10.$$

La primera cantidad toma como referencia **todos los spam**. Para clasificar necesitamos tomar como referencia los correos **con oferta**.

$$\mathrm{P}(\text{spam} \mid \text{oferta}) = \frac{16}{16+8} = \frac{2}{3} \approx 0.6667.$$

**El 80 % de los spam contiene “oferta”, pero sólo el 66.7 % de los correos con “oferta” es spam.**

# Bayes recupera el mismo cálculo

$Y = 1$: **spam**; $Y = 0$: correo legítimo. Aplicamos Bayes:

$$\mathrm{P}(\text{spam} \mid \text{oferta}) = \frac{\mathrm{P}(\text{spam})\ \mathrm{P}(\text{oferta} \mid \text{spam})}{\mathrm{P}(\text{legítimo})\ \mathrm{P}(\text{oferta} \mid \text{legítimo}) + \mathrm{P}(\text{spam})\ \mathrm{P}(\text{oferta} \mid \text{spam})}.$$

Reemplazamos con las frecuencias de la tabla:

$$\mathrm{P}(\text{spam} \mid \text{oferta}) = \frac{\underbrace{0.20}_{20/100} \times \underbrace{0.80}_{16/20}}{\underbrace{0.80}_{80/100} \times \underbrace{0.10}_{8/80} + \underbrace{0.20}_{20/100} \times \underbrace{0.80}_{16/20}}$$

$$= \frac{0.16}{0.08 + 0.16} = \frac{0.16}{0.24} \approx 0.6667.$$

**El denominador suma las dos formas de observar un correo con “oferta”.**

# Convertir los productos en probabilidades

Seguimos observando sólo la palabra *oferta*.

| Clase | Producto: clase y oferta | Dividir por el total |
|---|---|---|
| Legítimo | $(80/100)(8/80) = 0.08$ | $0.08/0.24 \approx 0.3333$ |
| Spam | $(20/100)(16/20) = 0.16$ | $0.16/0.24 \approx 0.6667$ |

$$\underbrace{0.24}_{\text{total con oferta}} = \underbrace{0.08}_{\text{legítimos con oferta}} + \underbrace{0.16}_{\text{spam con oferta}} .$$

**Normalizar significa dividir cada producto por la suma de ambos. Así las dos probabilidades suman uno.**

# La frecuencia de spam cambia la respuesta

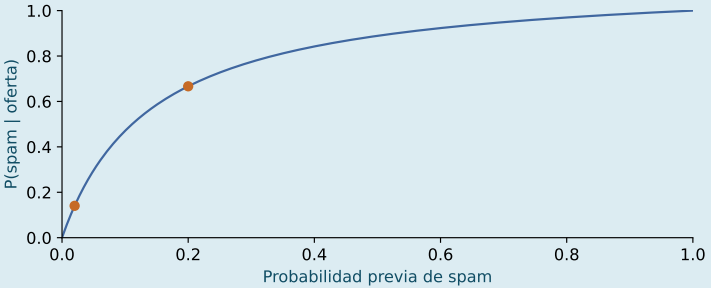
Mantenemos $P(\text{oferta} \mid \text{spam}) = 0.80$ y $P(\text{oferta} \mid \text{legítimo}) = 0.10$.

| Probabilidad | Ejemplo inicial | Nuevo escenario |
|---|---|---|
| $P(\text{spam})$ | $20/100 = 0.20$ | 0.02 (supuesto) |
| $P(\text{legítimo})$ | $1 - 0.20 = 0.80$ | $1 - 0.02 = 0.98$ |
| Spam y oferta | $0.20 \times 0.80 = 0.16$ | $0.02 \times 0.80 = 0.016$ |
| Legítimo y oferta | $0.80 \times 0.10 = 0.08$ | $0.98 \times 0.10 = 0.098$ |
| $P(\text{spam} \mid \text{oferta})$ | $\frac{0.16}{0.08 + 0.16} \approx 0.6667$ | $\frac{0.016}{0.098 + 0.016} \approx 0.1404$ |

**Con “oferta”:** $20\,\% \rightarrow 66.7\,\%$ **en el primer caso;** $2\,\% \rightarrow 14.0\,\%$ **en el segundo.**

En la última fila dividimos spam con oferta por todos los correos con oferta. El 2 % define un escenario hipotético.

# La misma palabra, dos puntos de partida




**Horizontal:** frecuencia previa de spam.

**Vertical:** probabilidad de spam al observar *oferta*.

Los puntos repiten la tabla anterior: 20 % $\rightarrow$ 66.7 % y 2 % $\rightarrow$ 14.0 %.

**La palabra ayuda, pero no decide por sí sola. Volvamos al ejemplo inicial y agreguemos otra señal del correo.**

# Dos señales en un mismo correo

Agregamos el indicador “enlace” al ejemplo de 100 correos.

| Clase | Correos | Con oferta | Con enlace |
|---|---|---|---|
| Legítimo | 80 | 8 | 16 |
| Spam | 20 | 16 | 14 |

Queremos clasificar un correo con $x = (1, 0)$: **oferta presente y enlace ausente**.

**Sabemos cuánto aparece cada señal por separado. Para clasificar este correo necesitamos combinarlas dentro de cada clase.**

# Presente o ausente: aquí aparece Bernoulli

Cada señal responde una pregunta con dos respuestas: ¿aparece *oferta*?, ¿hay enlace?

Una variable **Bernoulli** representa precisamente un resultado sí/no: usamos 1 para presente y 0 para ausente.

| En los 20 spam del ejemplo | Conteo | Probabilidad |
|---|---|---|
| Enlace presente (1) | 14 | $14/20 = 0.70$ |
| Enlace ausente (0) | $20 - 14 = 6$ | $6/20 = 0.30$ |

**Naive Bayes Bernoulli combina estas preguntas de presencia o ausencia para clasificar un correo.**

Un cero significa ausencia observada de la señal; no significa dato faltante.

# ¿Podemos contar todas las combinaciones?

Con dos señales, necesitamos distinguir cuatro tipos de correo:

| Oferta | Enlace |
|---|---|
| No | No |
| No | Sí |
| Sí | No |
| Sí | Sí |

Cada señal nueva duplica las combinaciones. Con diez señales hay $2^{10} = 1024$ patrones posibles por clase.

**Con pocos correos, muchos patrones quedan sin observar. Naive Bayes propone combinar lo que aprendimos de cada señal por separado.**

# Naive Bayes combina señales dentro de cada clase

El modelo supone que, **dentro de spam y dentro de legítimos**, las señales son independientes. Así podemos multiplicar sus probabilidades.

$$\mathrm{P}(\text{oferta, sin enlace} \mid \text{spam}) = \mathrm{P}(\text{oferta} \mid \text{spam})\ \mathrm{P}(\text{sin enlace} \mid \text{spam}).$$

- Repetimos el producto dentro de los correos legítimos.
- Cada señal aporta el estado observado: aquí, oferta presente y enlace ausente.
- Con más variables, incorporamos un factor por variable.

**Esta simplificación da el nombre *naive*. Hace posible el cálculo; no demuestra que las señales sean independientes en los datos.**

# Calcular el patrón que realmente observamos

El correo tiene **oferta y no tiene enlace**. Recuperamos los conteos de las dos clases:

$$\mathrm{P}(\text{oferta, sin enlace} \mid \text{spam}) = \underbrace{\frac{16}{20}}_{\text{oferta}} \times \underbrace{\frac{20-14}{20}}_{\substack{\text{sin enlace:} \\ \text{6 de 20 spam}}} = 0.80 \times 0.30 = 0.24.$$

$$\mathrm{P}(\text{oferta, sin enlace} \mid \text{legítimo}) = \underbrace{\frac{8}{80}}_{\text{oferta}} \times \underbrace{\frac{80-16}{80}}_{\substack{\text{sin enlace:} \\ \text{64 de 80 legítimos}}} = 0.10 \times 0.80 = 0.08.$$

**Estos productos describen el patrón dentro de cada clase. Para obtener la probabilidad de spam falta incorporar las frecuencias de clase y normalizar.**

Cálculo bajo independencia condicional; no son conteos observados del patrón completo.

# Preparar los factores del correo nuevo

Queremos clasificar: **oferta presente, enlace ausente**.

| Clase | Frecuencia de clase | Oferta presente | Enlace ausente |
|---|---|---|---|
| Legítimo | $80/100 = 0.80$ | $8/80 = 0.10$ | $(80 - 16)/80 = 0.80$ |
| Spam | $20/100 = 0.20$ | $16/20 = 0.80$ | $(20 - 14)/20 = 0.30$ |

La ausencia de enlace es más frecuente entre legítimos: 64 de 80, frente a 6 de 20 spam.

**En cada columna buscamos la probabilidad de lo que efectivamente observamos en el correo.**

# Oferta presente, enlace ausente

$s_0$: producto para **legítimo**. $s_1$: producto para **spam**.

$$s_0 = \underbrace{0.80}_{\substack{\text{clase legítima} \\ 80/100}} \times \underbrace{0.10}_{\substack{\text{oferta presente} \\ 8/80}} \times \underbrace{0.80}_{\substack{\text{enlace ausente} \\ 64/80}} = 0.064$$

$$s_1 = \underbrace{0.20}_{\substack{\text{clase spam} \\ 20/100}} \times \underbrace{0.80}_{\substack{\text{oferta presente} \\ 16/20}} \times \underbrace{0.30}_{\substack{\text{enlace ausente} \\ 6/20}} = 0.048$$

$$\text{P}(\text{spam} \mid \text{oferta, sin enlace}) = \frac{s_1}{s_0 + s_1} = \frac{0.048}{0.064 + 0.048} \approx 0.4286.$$

**Los productos $s_0$ y $s_1$ aún no son las probabilidades de clase del correo. Primero debemos normalizarlos.**

# ¿Qué cambia cuando vemos el enlace?

| Lo que observamos | Cálculo de la probabilidad de spam | Resultado |
|---|---|---|
| Sólo oferta | $0.16/(0.08 + 0.16)$ | 0.6667 |
| Oferta, sin enlace | $0.048/(0.064 + 0.048)$ | 0.4286 |
| Oferta y enlace | $0.112/(0.016 + 0.112)$ | 0.8750 |

En la última fila, oferta y enlace están presentes. Usamos sus frecuencias dentro de cada clase:

$$\underbrace{\tfrac{20}{100} \times \tfrac{16}{20} \times \tfrac{14}{20} = 0.112}_{\text{spam y ambas señales}}, \qquad \underbrace{\tfrac{80}{100} \times \tfrac{8}{80} \times \tfrac{16}{80} = 0.016}_{\text{legítimo y ambas señales}}.$$

**No observar el enlace no es lo mismo que desconocer si existe. Con umbral 0.50, su ausencia cambia la decisión a legítimo.**

# ¿No apareció o es imposible que aparezca?

Agreguemos una palabra hipotética, *premio*, que no apareció en ninguno de los 20 spam del ejemplo:

$$\mathrm{P}(\text{premio} \mid \text{spam}) = \frac{0}{20} = 0.$$

Si el próximo correo contiene *premio*, multiplicar por cero anula el producto para spam, aunque las demás señales favorezcan esa clase.

Observar cero casos en una muestra pequeña no demuestra que el evento sea imposible.

**Necesitamos evitar que un conteo cero cierre por completo una posibilidad.**

# Suavizar: dar un pequeño peso a ambas respuestas

Con suavizado de Laplace, agregamos **uno a presente y uno a ausente**. Para *premio* entre los spam:

$$\mathrm{P}(\text{premio} \mid \text{spam}) = \frac{\overbrace{0}^{\text{presencias observadas}} + \overbrace{1}^{\text{peso agregado}}}{\underbrace{20}_{\text{correos spam}} + \underbrace{2}_{\text{uno por cada respuesta}}} = \frac{1}{22} \approx 0.0455.$$

Llamamos $\alpha$ al peso agregado a cada respuesta. Aquí usamos $\alpha = 1$; otros valores pueden compararse mediante validación.

**El denominador aumenta en dos porque existen dos respuestas posibles: presente y ausente.**

El suavizado modifica las probabilidades de las señales. Conservamos las frecuencias de clase: 80/100 y 20/100.

# Reconstruir los factores con suavizado

Correo con **oferta presente y enlace ausente**; $\alpha = 1$.

| Clase | Oferta presente | Enlace ausente |
|---|---|---|
| Legítimo | $\frac{8+1}{80+2} = \frac{9}{82}$ | $\frac{(80-16)+1}{80+2} = \frac{65}{82}$ |
| Spam | $\frac{16+1}{20+2} = \frac{17}{22}$ | $\frac{(20-14)+1}{20+2} = \frac{7}{22}$ |

$$\frac{7}{22} = \frac{\underbrace{6}_{\text{spam sin enlace}} + \underbrace{1}_{\text{suavizado}}}{\underbrace{20}_{\text{spam observados}} + \underbrace{2}_{\text{dos respuestas}}}.$$

**Spam sin enlace:** $20 - 14 = 6$**. Con suavizado:** $6 + 1 = 7$ **sobre** $20 + 2 = 22$**.**

# Completar el cálculo con suavizado

Usamos los factores de la tabla anterior y las mismas frecuencias de clase:

$$s_0 = \underbrace{0.80}_{\text{legítimo}} \times \underbrace{\frac{9}{82}}_{\text{oferta}} \times \underbrace{\frac{65}{82}}_{\text{sin enlace}} \approx 0.0696$$

$$s_1 = \underbrace{0.20}_{\text{spam}} \times \underbrace{\frac{17}{22}}_{\text{oferta}} \times \underbrace{\frac{7}{22}}_{\text{sin enlace}} \approx 0.0492$$

$$\mathrm{P}(\text{spam} \mid \text{oferta, sin enlace}) = \frac{s_1}{s_0 + s_1} \approx 0.4140.$$

**Pasamos de 0.4286 a 0.4140. El suavizado evita ceros; no corrige la dependencia entre señales.**

El resultado se calcula con las fracciones completas, sin redondear los productos intermedios.

# Separar lo que aprendemos de lo que predecimos

**Ajustar con correos etiquetados**
Contamos cuántos correos hay en cada clase y cuántos presentan cada señal.

Con esos conteos y el suavizado obtenemos las probabilidades que usará el modelo.

**Predecir un correo**
Leemos qué señales están presentes y cuáles ausentes, sin conocer su etiqueta.

Elegimos los factores correspondientes, multiplicamos por la previa y normalizamos.

**Las probabilidades se aprenden una vez durante el ajuste. Después podemos utilizarlas para clasificar distintos correos.**

# El mismo cálculo en R y Python

Oferta presente y enlace ausente; suavizado $\alpha = 1$.

**Python**

```
s_legitimo = (80/100)*(9/82)*(65/82)
s_spam = (20/100)*(17/22)*(7/22)
p_spam = s_spam/(s_legitimo+s_spam)
print(p_spam)
```

**R**

```
s_legitimo <- (80/100)*(9/82)*(65/82)
s_spam <- (20/100)*(17/22)*(7/22)
p_spam <- s_spam/(s_legitimo+s_spam)
print(p_spam)
```

Salida en ambos lenguajes: $p_{\text{spam}} \approx 0.4140$.

**Ambos lenguajes reproducen el cálculo que hicimos a mano.**

# Leer la salida sin perder los conteos

| Probabilidad | Sin suavizado | Con $\alpha = 1$ |
|---|---|---|
| Oferta entre spam | $16/20 = 0.8000$ | $17/22 \approx 0.7727$ |
| Enlace entre spam | $14/20 = 0.7000$ | $15/22 \approx 0.6818$ |
| Spam para oferta sin enlace | 0.4286 | 0.4140 |

Las primeras filas describen señales dentro de una clase. La última clasifica el correo combinando **ambas clases** y la **ausencia** de enlace.

**En el CSV, *theta* identifica una probabilidad de presencia por clase; *posterior* es la probabilidad de spam del correo nuevo.**

# El programa aprende las mismas probabilidades

Filas: legítimo y spam; columnas: oferta y enlace. Con $\alpha = 1$:

**Python**

```
n = np.array([80., 20.])
conteos = np.array([[8., 16.],
                    [16., 14.]])
alfa = 1
prior = n / n.sum()
theta = (conteos + alfa) / (
    n[:, None] + 2*alfa)
print(theta)
```

**R**

```
n <- c(80, 20)
conteos <- rbind(c(8, 16),
                 c(16, 14))
alfa <- 1
prior <- n / sum(n)
theta <- (conteos + alfa) /
         (n + 2*alfa)
print(theta)
```

Salida para spam: $(17/22, 15/22) \approx (0.7727, 0.6818)$. La previa es $(0.80, 0.20)$.

**Cada celda estima una presencia por clase. Para una ausencia usamos $1 -$ `theta`.**

Fragmento del cálculo de los scripts. Python utiliza numpy, importado como np en el entorno inicial.

# Cada salida del modelo responde una pregunta

Para el correo con oferta y sin enlace, usando $\alpha = 1$:

| Cantidad | Qué significa | En nuestro ejemplo |
|---|---|---|
| Entrada | Señales que observamos en el correo | Oferta sí, enlace no |
| Previa | Frecuencia de clase antes de leerlo | $P(\text{spam}) = 0.20$ |
| Condicional | Frecuencia de una señal dentro de clase | Oferta en spam: $17/22$ |
| Posterior | Probabilidad de clase al combinar las señales | 0.4140 de spam |

**El 0.7727 describe oferta dentro de spam. El 0.4140 describe spam para este correo: son preguntas diferentes.**

# La misma cuenta para varias clases

La cuenta anterior también sirve para $K$ clases $c_1, \ldots, c_K$. Por ejemplo: spam, personal y trabajo. Para un correo con entradas $x$:

- $\pi_k = \mathrm{P}(Y = c_k)$: probabilidad previa de la clase $c_k$.
- $f_k(x)$: probabilidad de las entradas $x$ dentro de esa clase; para entradas continuas, su densidad.

$$\mathrm{P}(Y = c_k \mid X = x) = \frac{\underbrace{\pi_k}_{\text{previa}} \times \underbrace{f_k(x)}_{\text{entradas en esa clase}}}{\underbrace{\sum_{\ell=1}^{K} \pi_\ell f_\ell(x)}_{\text{suma de los productos de todas las clases}}}.$$

El índice $\ell$ recorre las clases: con tres clases, el denominador suma tres productos.

**Normalizamos igual que con spam y legítimo, ahora entre $K$ clases.**

# Naive Bayes: un factor por variable

Tenemos $d$ variables, $x = (x_1, \ldots, x_d)$; $f_{jk}(x_j)$ es el factor del valor observado de la variable $j$ en la clase $c_k$.

Por independencia dentro de clase, $f_k(x) = \prod_{j=1}^{d} f_{jk}(x_j)$:

$$\mathrm{P}(Y = c_k \mid X = x) = \frac{\pi_k \underbrace{\prod_{j=1}^{d} f_{jk}(x_j)}_{\text{multiplicar los factores de las variables}}}{\sum_{\ell=1}^{K} \pi_\ell \prod_{j=1}^{d} f_{j\ell}(x_j)}.$$

Con oferta y enlace, $d = 2$: el numerador para spam es $0.20(17/22)(7/22)$.

**El producto recorre las variables; la suma recorre las clases.**

# Ya construimos una probabilidad: falta ponerla a prueba

**Para el correo con oferta y sin enlace**, los conteos de 100 correos y el suavizado $\alpha = 1$ dieron $P(\text{spam} \mid x) \approx 0.4140$.

Sabemos qué representa cada factor y cómo R y Python repiten el cálculo. Con umbral 0.50, el mensaje se clasifica como legítimo.

**Al volver a los datos reales**, aplicaremos la receta a 54 señales y comprobaremos sus probabilidades en correos que no participaron en el ajuste.

**La segunda parte responderá las preguntas pendientes: qué tan fiables son esas probabilidades y cuándo conviene bloquear.**

# Construyamos el filtro con 4601 correos

Aplicaremos la receta a los correos etiquetados de **Spambase**.

| Qué contiene | Qué utilizaremos | Para qué |
|---|---|---|
| 48 frecuencias de palabras | Las 48 columnas | Describir el contenido |
| 6 frecuencias de caracteres | Las 6 columnas | Describir otras señales |
| Etiqueta de cada correo | Spam o legítimo | Aprender por clase |

Usamos **54 frecuencias**; excluimos las tres medidas de mayúsculas.

**Dadas las señales de un correo, ¿qué probabilidad de spam le asigna el modelo?**

Hopkins et al. (1999), Spambase, UCI, DOI 10.24432/C53G6X; CC BY 4.0. Corpus histórico.

# Una fila describe las señales de un correo

El **correo ID 1** tiene el menor identificador de desarrollo. Mostramos tres de sus 54 frecuencias:

| Señal | Frecuencia registrada | Presencia: frecuencia $> 0$ |
|---|---|---|
| Palabra `free` | 0.32 % de las palabras | 1: presente |
| Palabra `money` | 0 % de las palabras | 0: ausente |
| Carácter `!` | 0.778 % de los caracteres | 1: presente |

El CSV guarda 0.32 para representar 0.32 %. Bernoulli utiliza el indicador de presencia mostrado en la última columna.

**La predicción utiliza las señales. El identificador y la etiqueta del correo no son entradas del modelo.**

Valores del archivo local `spam_s40.csv`. Caso elegido por identificador, sin consultar su clase.

# Leer los datos y separar entradas de respuesta

En los scripts, `DATOS` es la carpeta del CSV. El entorno de Python ya dispone de pandas como `pd`.

**Python**

```python
datos = pd.read_csv(DATOS/"spam_s40.csv")
variables = [c for c in datos
    if c.startswith(("word_freq_",
                     "char_freq_"))]
ent = datos[
    datos.conjunto == "entrenamiento"]
X = ent[variables].to_numpy()
y = ent.spam.to_numpy()
```

**R**

```r
datos <- read.csv(
  file.path(DATOS,"spam_s40.csv"),
  check.names=FALSE)
variables <- grep("^(word_freq_|char_freq_)",
  names(datos), value=TRUE)
ent <- datos[
  datos$conjunto=="entrenamiento", ]
X <- as.matrix(ent[,variables])
y <- ent$spam
```

`X` contiene 3680 correos y 54 frecuencias; `y`, sus clases (1: spam; 0: legítimo).

**Seleccionar las columnas por su nombre deja fuera identificadores, grupos, pliegues y la propia respuesta.**

# Aprender con unos correos y predecir otros

Conservamos la partición guardada en los datos:

| Conjunto | Correos | Uso |
|---|---|---|
| Desarrollo | 3680 | Aprender y comparar recetas |
| Prueba histórica | 921 | Comprobar la receta elegida |

En **cinco pliegues**, aprendemos con cuatro y predecimos el restante. Al repetirlo, cada correo recibe una predicción **fuera de pliegue (OOF)**.

Los 921 correos ya se usaron al evaluar otros clasificadores: la comprobación es histórica, no una nueva prueba independiente.

**Compararemos recetas con predicciones de correos excluidos de su respectivo ajuste.**

Las filas con las mismas 57 entradas originales se mantienen juntas. Estos grupos no identifican remitentes ni fechas.

# Del correo real a su probabilidad de spam

La función `ajustar_predecir`, incluida en ambos scripts, recibe datos de ajuste y correos por predecir. Usamos Bernoulli con $\alpha = 1$.

**Python**

```python
i = int(np.argmin(ent.id.to_numpy()))
correo = X[i:i+1]
p, modelo, logp = ajustar_predecir(
    X, y, correo, "bern", 1)
print(ent.id.iloc[i])
print(p[0])
```

**R**

```r
i <- which.min(ent$id)
correo <- X[i,,drop=FALSE]
ajuste <- ajustar_predecir(
  X, y, correo, "bern", 1)
print(ent$id[i])
print(ajuste$p[1])
```

Ambos devuelven ID 1 y probabilidad de spam $\widehat{p} \approx 0.9964$. La función transforma cada frecuencia positiva en presencia y combina las 54 señales.

**Este correo participa en el ajuste: el cálculo ilustra una predicción. Evaluaremos la calidad del filtro con las probabilidades OOF.**

# ¿Basta con saber que una palabra está presente?

La frecuencia de `free` en ID 1 es 0.32 %. Dos representaciones posibles:

| Representación | Valor que entra | Información conservada |
|---|---|---|
| Presencia | 1, porque $0.32 > 0$ | La palabra aparece |
| Frecuencia | 0.32 | Qué proporción de palabras corresponde a `free` |

Bernoulli trata igual cualquier frecuencia positiva. Conservar el valor permite distinguir cuánto aparece la palabra.

**Conservar más detalle exige otro supuesto sobre los datos. Comprobaremos si ese detalle mejora las probabilidades.**

# La variante gaussiana describe valores con curvas

Naive Bayes gaussiano describe los valores de cada variable con una **curva normal dentro de cada clase**.

**Qué aprende**
Una **media**, que sitúa el centro de la curva, y una **dispersión**, que determina su amplitud, para cada variable y clase.

**Cómo predice**
Evalúa las alturas en los valores del correo. Multiplica esas alturas y la previa de clase; luego normaliza.

La altura es una **densidad**, no la probabilidad de un valor exacto. Se conserva la independencia entre variables dentro de clase.

**Evaluaremos si esta aproximación normal produce buenas probabilidades para los correos.**

# Transformar la frecuencia antes de ajustar

Las frecuencias son no negativas y algunas pueden ser grandes. En la receta gaussiana usaremos $z = \log(1 + x)$, con logaritmo natural.

| Señal del correo ID 1 | Valor $x$ en el CSV | Valor transformado $z$ |
|---|---|---|
| `free` | 0.32 | $\log(1.32) \approx 0.2776$ |
| `money` | 0 | $\log(1) = 0$ |

La transformación comprime valores grandes y conserva los ceros. Las medias y dispersiones se aprenden en esta **escala transformada**.

**Transformar no garantiza normalidad ni independencia. La comparación OOF mostrará cómo funciona esta receta en los correos disponibles.**

Python y R usan `log1p(x)` para calcular $\log(1 + x)$ con precisión numérica.

# Contar repeticiones requiere el texto

Con los mensajes disponibles, fijamos un vocabulario y contamos las apariciones:

| Mensaje | oferta | reunion | hoy |
|---|---|---|---|
| *oferta hoy* | 1 | 0 | 1 |
| *oferta oferta hoy* | 2 | 0 | 1 |

**Bernoulli**: ambos mensajes tienen las presencias $(1, 0, 1)$. **Multinomial**: cada aparición aporta un factor; la repetición sí cambia el cálculo.

Multinomial aprende proporciones de términos contando palabras de cada clase. La previa se estima contando documentos.

**Multinomial conserva las repeticiones; Bernoulli conserva sólo las presencias.**

Mensajes construidos para ilustrar las representaciones; no proceden de Spambase.

# La información disponible determina qué comparar

Volvamos a las 54 frecuencias del archivo local:

| Variante | Entrada que necesita | Decisión para este caso |
|---|---|---|
| Bernoulli | Presencias o ausencias | Usar frecuencia $> 0$ |
| Gaussiana | Mediciones numéricas | Probar $\log(1 + \text{frecuencia})$ |
| Multinomial | Conteos de términos | Requiere información adicional |

Un porcentaje no permite recuperar cuántas veces apareció una palabra sin conocer la longitud del mensaje. No trataremos estas frecuencias como conteos de palabras.

**Compararemos Bernoulli y la receta gaussiana sobre los mismos correos y pliegues. La siguiente pregunta es cuál produce mejores probabilidades.**

# Qué recetas compararemos y cómo elegiremos

Usamos las mismas 54 frecuencias y los mismos cinco pliegues de desarrollo.

| Receta | Qué aprende | Opciones |
|---|---|---|
| Frecuencia de spam | Una probabilidad sin usar señales | Una referencia |
| Bernoulli | Presencias y ausencias | $\alpha = 0.5,\ 1,\ 2$ |
| Gaussiana | Medias y dispersiones de $\log(1 + x)$ | Una receta |

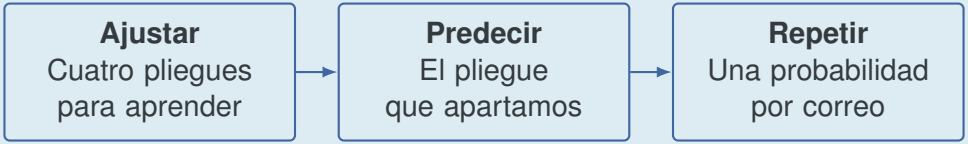
Son **cinco candidatos**: una referencia, tres suavizados y una gaussiana.

1. Elegimos el menor **log loss**: penaliza asignar poca probabilidad a la clase que realmente ocurrió.
2. Después elegimos el **umbral** del ganador: queremos una regla de bloqueo.

**Primero comparamos las probabilidades; después decidimos qué hacer con ellas.**

# Una predicción sin participar en su ajuste

Los 3680 correos de desarrollo están repartidos en cinco pliegues.




Repetimos cinco veces y reunimos las probabilidades **OOF** (*out of fold*, fuera de pliegue). Las comparamos con las etiquetas reales.

Cinco recetas y cinco pliegues requieren $5 \times 5 = 25$ ajustes.

**La fila que recibe una predicción no aportó al ajuste que la produjo.**

La selección usa desarrollo; los 921 correos de prueba histórica se mantienen aparte.

# El mismo recorrido de validación en R y Python

Para Bernoulli con $\alpha = 1$, `val` identifica el pliegue que apartamos:

**Python**

```python
pred = np.empty(len(y))
for k in sorted(np.unique(folds)):
    val = folds == k
    pred[val], _, _ = ajustar_predecir(
        X[~val], y[~val], X[val],
        "bern", 1.)
```

**R**

```r
pred <- numeric(length(y))
for (k in sort(unique(folds))) {
  val <- folds == k
  ajuste <- ajustar_predecir(
    X[!val, ], y[!val], X[val, ],
    "bern", 1)
  pred[val] <- ajuste$p
}
```

`X`: 54 frecuencias; `y`: etiquetas; `folds`: pliegue de cada correo. `pred` reúne las 3680 probabilidades OOF de spam.

**Repetimos este recorrido para cada receta. El script conserva también las log-probabilidades para evaluar con precisión.**

# Bernoulli obtiene el menor error probabilístico

**Predicciones OOF de los mismos 3680 correos.**

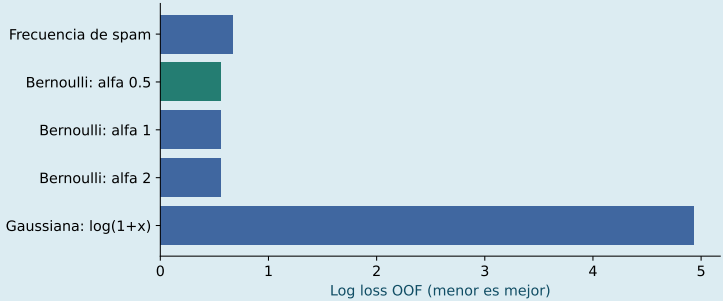
| Receta | Log loss | Brier | AUC |
|---|---|---|---|
| Frecuencia de spam | 0.671 | 0.239 | 0.499 |
| Bernoulli, $\alpha = 0.5$ | 0.556 | 0.099 | 0.952 |
| Bernoulli, $\alpha = 1$ | 0.557 | 0.099 | 0.951 |
| Bernoulli, $\alpha = 2$ | 0.558 | 0.100 | 0.951 |
| Gaussiana, $\log(1 + x)$ | 4.931 | 0.190 | 0.940 |

Brier promedia el error cuadrático entre probabilidad y etiqueta (0 o 1): menor es mejor. AUC mide el ordenamiento entre clases: mayor es mejor.

**Elegimos Bernoulli con $\alpha = 0.5$: log loss $0.556$. Los tres suavizados dan resultados cercanos.**

El mínimo OOF intervino en la selección; no constituye una evaluación independiente del ganador.

# Los aciertos pueden esconder certeza equivocada




La gaussiana acierta el **80.8 %** con umbral 0.5, pero su log loss es **4.931**, frente a 0.556 de Bernoulli.

**El log loss penaliza los errores con certeza excesiva. El porcentaje de aciertos no muestra esa diferencia.**

# Copiar una señal aumenta la certeza del modelo

Experimento controlado: **100 spam y 100 legítimos**; una señal aparece en 80 spam y 20 legítimos.

$$\mathrm{P}(\text{spam} \mid \text{señal}) = \frac{\underbrace{0.5}_{100/200} \times \underbrace{0.8}_{80/100}}{0.5 \times 0.8 + 0.5 \times \underbrace{0.2}_{20/100}} = 0.8000.$$

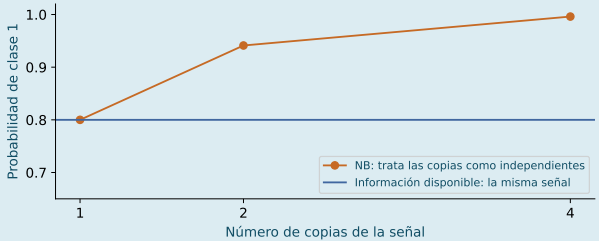
Copiamos esa misma columna. Naive Bayes multiplica las dos copias:

$$\widehat{p} = \frac{0.5 \times 0.8 \times 0.8}{0.5 \times 0.8 \times 0.8 + 0.5 \times 0.2 \times 0.2} \approx 0.9412.$$

**El correo y su información son los mismos. El modelo contó dos veces una señal y aumentó su certeza.**

Ejemplo construido sobre dependencia; no explica por sí solo los resultados de Spambase.

# La clase se mantiene; la certeza sigue creciendo

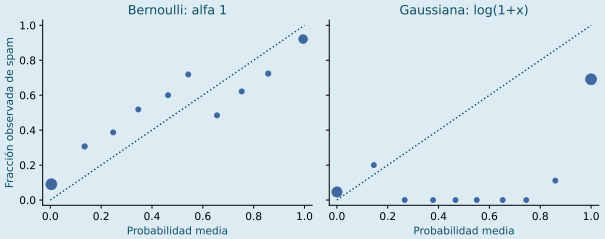



Cuatro copias repiten cuatro veces los factores del ejemplo anterior:

$$\widehat{p} = \frac{0.5(0.8)^4}{0.5(0.8)^4 + 0.5(0.2)^4} \approx 0.9961.$$

**Las tres versiones predicen spam a umbral 0.5. Ninguna copia aporta información que justifique pasar de 80 % a 99.6 %.**

# Contrastar la probabilidad con lo que ocurrió




**Bernoulli,** $\alpha = 1$
Intervalo 0.9–1:
1157 correos.

Predicción media: **99.44 %**.
Spam observado:

$$\frac{1066}{1157} \approx 92.13\,\%.$$

Ejes: probabilidad media y fracción de spam. Tamaño: número de correos; intervalos de amplitud 0.1.

**En ese grupo, 99.44 % predicho supera 92.13 % observado. La diagonal marca coincidencia entre probabilidad y frecuencia.**

Diagnóstico OOF: Bernoulli usa $\alpha = 1$; el ganador de la selección usa $\alpha = 0.5$. No se ajustó un calibrador.

# Reconstruir una señal del correo real

Correo **ID 1**: `address` presente. Conteos de desarrollo; $\alpha = 1$:

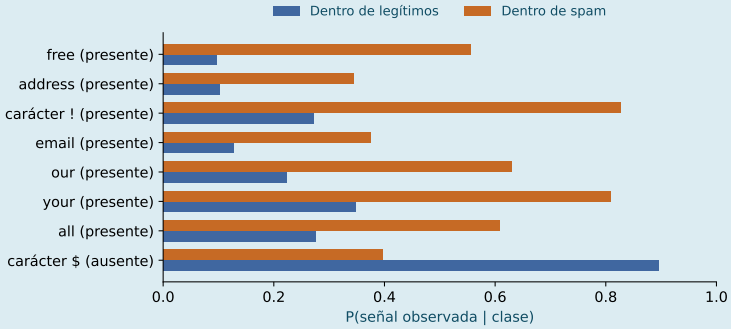
| Clase | Presencias | Total | Probabilidad suavizada |
|---|---|---|---|
| Legítimo | 226 | 2230 | $\frac{226+1}{2230+2} \approx 0.1017$ |
| Spam | 498 | 1450 | $\frac{498+1}{1450+2} \approx 0.3437$ |

Suavizado: uno por presencia y otro por ausencia; el total aumenta en dos.

**Address presente favorece spam:** $0.3437 > 0.1017$**. Aún falta combinar las otras 53 señales.**

Son probabilidades de una señal dentro de cada clase, no probabilidades posteriores de spam.

# Combinar las señales de un correo real




**Ocho señales del correo**
Cada par compara la probabilidad del estado observado dentro de cada clase.

**Predicción completa**
Las 54 señales y las frecuencias de clase producen $\widehat{p}_{\text{spam}} = 0.9964$.

**La predicción combina presencias y ausencias. Las alturas de las barras son factores: no se suman ni representan efectos causales.**

ID 1, el menor de desarrollo; Bernoulli con $\alpha = 1$. El correo participó en el ajuste: es explicación del modelo, no evaluación OOF.

# ¿Cuándo conviene bloquear un correo?

Un **falso positivo (FP)** bloquea un legítimo (costo 5); un **falso negativo (FN)** deja pasar spam (costo 1). Los aciertos cuestan cero.

| Probabilidad | Bloquear | Dejar pasar | Menor costo |
|---|---|---|---|
| $p = 0.90$ | $5(1 - 0.90) = 0.50$ | $1(0.90) = 0.90$ | Bloquear |
| $p = 0.70$ | $5(1 - 0.70) = 1.50$ | $1(0.70) = 0.70$ | Dejar pasar |

Si $p$ es la probabilidad de spam, bloquear arriesga el costo de un legítimo $(1 - p)$; dejar pasar, el de un spam $(p)$.

**Como la calibración es imperfecta, compararemos umbrales con los errores observados.**

Costos y probabilidades hipotéticos para explicar la decisión; la búsqueda siguiente usa predicciones OOF.

# Buscar el umbral en las predicciones del ganador

La función `medir` cuenta FP y FN y calcula $1000(5\,\mathrm{FP} + \mathrm{FN})/n$, el costo por 1000 correos.

**Python**

```
umbrales = np.arange(1, 20)/20
costos = pd.DataFrame([
    dict(umbral=t, **medir(
        y, oof[ganador], t))
    for t in umbrales])
elegido = costos.sort_values(
    ["costo_1000", "umbral"],
    ascending=[True, False]).iloc[0]
```

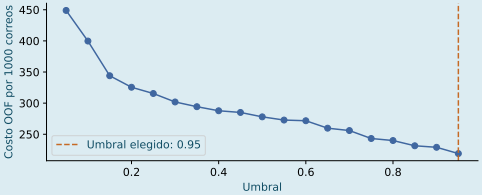
**R**

```
umbrales <- (1:19)/20
costos <- do.call(rbind,
  lapply(umbrales, function(t)
    cbind(umbral=t, medir(
      y, oof[[ganador]], t))))
elegido <- costos[order(
  costos$costo_1000,-costos$umbral),][1,]
```

Usamos las probabilidades OOF del Bernoulli seleccionado ($\alpha = 0.5$).

**Entre 19 umbrales, de 0.05 a 0.95, elegimos 0.95. Si dos costos empatan, preferimos el umbral mayor.**

# El umbral elegido cambia los errores del filtro




Con $t = 0.95$: **78 legítimos bloqueados** y **416 spam que pasan**, entre 3680 correos.

$$\text{Costo por } 1000 = \frac{5 \times \underbrace{78}_{\text{FP}} + \underbrace{416}_{\text{FN}}}{3680} \times 1000 = 219.0.$$

**0.95 tiene el menor costo entre los 19 umbrales. Estar en el borde no demuestra que sea el mejor umbral posible.**

Búsqueda sobre probabilidades OOF del ganador; no se usaron los correos de prueba histórica para elegir.

# Qué obtiene la regla en los 921 correos históricos

| Real / predicha | Legítimo | Spam |
|---|---|---|
| Legítimo | 541 | 17 |
| Spam | 89 | 274 |

Filas: clase real. Columnas: decisión.

Bernoulli, $\alpha = 0.5$; umbral 0.95.

Log loss: 0.533. Brier: 0.094.

**Costo del clasificador**

17 legítimos bloqueados y 89 spam que pasan:

$$\frac{5 \times 17 + 89}{921} \times 1000 = 188.9.$$

**Si dejamos pasar todos**

Los $89 + 274 = 363$ spam pasan:

$$\frac{363}{921} \times 1000 = 394.1.$$

**La regla reduce el costo frente a dejar pasar todos. Es una comprobación retrospectiva: estos correos ya se habían utilizado.**

Modelo y umbral se eligieron en desarrollo. El resultado no acredita rendimiento en correos actuales.

# Qué revisar antes de usar el filtro en otra bandeja

| Lo que aprendimos | Qué revisar en la nueva bandeja |
|---|---|
| Con oferta: 66.7 % $\rightarrow$ 14.0 % al pasar de 20 % a 2 % de spam | Si la frecuencia de spam del ajuste sigue representando a los nuevos correos |
| Copiar una señal elevó la certeza sin agregar información | Dependencia entre señales y calibración de las probabilidades |
| Spambase contiene correos históricos | Resultados en correos actuales aún no utilizados |

**Cambiar o calibrar el filtro exige evaluar la receta completa con datos separados.**

# La regla que construimos, de los datos a la acción

| Paso | Decisión del caso de correos |
|---|---|
| Representar | 54 frecuencias convertidas en presencia o ausencia |
| Aprender | Frecuencias por clase con suavizado $\alpha = 0.5$ |
| Predecir | Combinar presencias y ausencias; normalizar entre las dos clases |
| Actuar | Bloquear cuando la probabilidad de spam sea al menos 0.95 |
| Comprobar | Contar errores y costo; revisar si las probabilidades reflejan frecuencias |

**Suavizado y umbral cumplen tareas distintas: uno modifica el cálculo de la probabilidad; el otro modifica la acción.**

# Reproducir el recorrido con los archivos de la sesión

1. Guardar juntos `spam_s40.csv` y el script del lenguaje elegido: `40_naive_bayes.R` o `40_naive_bayes.py`.
2. Ejecutar el script completo o sus secciones **00–10 en orden**, conservando la misma sesión de trabajo.
3. Leer las tablas y figuras que se generan: primero los ejemplos pequeños; después la comparación, el correo explicado y la regla de bloqueo.

Cada salida conserva los resultados para revisarlos sin repetir las cuentas a mano. No es necesario ejecutar ambos lenguajes para seguir el análisis.

**La presentación explica las decisiones; los scripts permiten reconstruir sus números con los mismos datos y pliegues.**

R usa funciones base. Python requiere numpy, pandas, matplotlib y scikit-learn. Los datos se leen localmente.

# ¿Qué significa una probabilidad de spam muy alta?

**En el correo explicado obtuvimos $\widehat{p} = 0.9964$. ¿Qué podemos concluir?**

- **Sí:** bajo ese ajuste Bernoulli, las señales del correo favorecen mucho a spam frente a legítimo.
- **Aún no:** que el 99.64 % de correos similares sea realmente spam; esa lectura requiere comprobar calibración.
- **Tampoco:** que el correo sea habitual. Un clasificador reparte probabilidad entre sus clases aunque el caso resulte extraño.

**Una probabilidad alta de clase expresa una preferencia del modelo. Su fiabilidad y la rareza del caso son preguntas adicionales.**

El correo ID 1 se explicó con $\alpha = 1$ y participó en el ajuste; no fue una predicción OOF.

# Referencias y material de la sesión


- Manning, C. D., Raghavan, P. y Schütze, H. (2008). *Introduction to Information Retrieval*, capítulo 13. Texto de los autores.
- scikit-learn. Naive Bayes y calibración de probabilidades. Fórmulas e implementación.
- Hopkins, M., Reeber, E., Forman, G. y Suermondt, J. (1999). *Spambase*. UCI. DOI 10.24432/C53G6X, CC BY 4.0.


Material: presentación, `40_naive_bayes.py`, `40_naive_bayes.R` y `spam_s40.csv`.

**Los scripts contienen cálculos, figuras y tablas. Pueden ejecutarse sin conexión una vez instaladas sus dependencias.**

# De observar una palabra a decidir qué hacer

**¿Basta con que un correo contenga oferta para bloquearlo?**

1. **Construir la probabilidad.** En el ejemplo de 100 correos, oferta sin enlace produjo $\widehat{p} \approx 0.4140$ con suavizado: también importan las ausencias y la frecuencia de clase.
2. **Comprobar su calidad.** En Spambase elegimos Bernoulli con $\alpha = 0.5$ por el menor log loss OOF entre las recetas comparadas.
3. **Decidir con un propósito.** Con costos de 5 por bloquear un legítimo y 1 por dejar pasar spam, elegimos umbral 0.95 en desarrollo.

**Una señal aporta información; Bayes la combina, la evaluación contrasta la probabilidad y el costo orienta la decisión.**

# Para continuar: Naive Bayes en R y Python

Las bibliotecas automatizan el ajuste y la predicción que construimos paso a paso.

| Variante | R: `naivebayes` | Python: `scikit-learn` |
|---|---|---|
| Bernoulli | `bernoulli_naive_bayes()` | `BernoulliNB()` |
| Gaussiana | `gaussian_naive_bayes()` | `GaussianNB()` |
| Multinomial | `multinomial_naive_bayes()` | `MultinomialNB()` |

**R: obtener probabilidades**

```
predict(modelo, nuevos, type="prob")
```

**Python: obtener probabilidades**

```
modelo.predict_proba(nuevos)
```

Bernoulli usa entradas binarias. Suavizado: `laplace` en R; `alpha` en Python.

**La biblioteca automatiza las cuentas; conservamos la representación, la validación y el umbral.**

Documentación: naivebayes (R) y Naive Bayes en scikit-learn (Python).